Last modification: March 17, 2026

RBE3002: Unified Robotics IV
D-Term 2026
Homework 0

Disclaimer

It might happen that you will find solutions, either partial or complete, on the web, on sites such as StackOverflow. Feel free to consult these websites, but

1. Remember that we *know* that these websites exist.
2. We have tools to detect if submitted work is not your own.
3. If you copy-paste code you don't understand, you're missing the point of this homework and the reason why it was given. **ROS will have the last laugh**.
4. You may **not** use automatic code-generation tools like Co-pilot, ChatGPT, and so forth.

Deliverables

The main deliverable of this homework is a `.zip` file containing three files: `ex1.py`, `ex2.py` and `ex3.py`. The files must be stored in a directory called `LastnameFirstnameM` (`M` is your middle initial if you have it). For example, if your name is *Donald J. Duck*, you'll submit a `.zip` file that contains:

```
DuckDonaldJ/
    ex1.py
    ex2.py
    ex3.py
```

The `.zip` filename must be called `RBE3002_HW1_LastnameFirstnameM.zip`. For example: `RBE3002_HW1_DuckDonaldJ.zip`. If your name contains symbols such as hyphens or apostrophes, please leave those out.

If you don't comply with these guidelines, **each mistake will drop 10% of the points you earned**. Example mistakes: file is not a `.zip`, but a `.rar` or a `.tar.gz`; once unzipped, there is no directory with your name; there are missing/extra files in the directory; the files are the right ones, but the names are wrong.

The names of the functions in the code skeletons provided below are mandatory — these functions must be implemented and named as indicated. You can write extra functions, if you think it's a good idea. Again, **each mistake in naming the functions will cost you 10% of your points**.

The reason why we want you follow these guidelines strictly is that, if you do, we'll be able to make the grading process almost completely automatic.

All the code must be written with Python 3.

Exercise 1: Loops, Arrays, Tuples (15 points)

A robot moves in a 2D lattice. The robot can only move north (`N`), south (`S`), east (`E`), and west (`W`) by a certain number of steps. Assume that the robot always starts at (0,0), and that the x axis is oriented east, while the y axis is oriented north. Write a Python function that, given a set of moves, calculates the final position of the robot.

For example, if the set of moves provided as input is:

```
N 2
E 4
S 1
W 3
```

then the final position of the robot is (1,1). The integer that follows a move is always positive.

Follow this code skeleton:

```python
def robot_move(moves):
    x = 0
    y = 0
    # your code here
    return (x,y)
```

The moves are provided as an array of tuples. The array can have any number of moves in any order. The example above is passed as input as follows:

```
[('N',2), ('E',4), ('S',1), ('W',3)]
```

Exercise 2: Recursion (20 points)

Write a *recursive* function that returns `True` if the numerical array passed as input is palindrome, and `False` otherwise. An array is *palidrome* if its elements, when flipped, produce an array that is identical to the original one. For example:

```
[1,2,3,2,1] is palindrome
[1,2,3,4,3] is not palindrome
```

Fill in the following code skeleton. It is composed of four parts: three base cases and the recursive call. Write code for all the parts. Assume that the input arrays can have any size, and that the elements are always positive integers.

```python
def ispalindrome(arr):
    # Base case 1:
    # An empty array is always palindrome -> return True

    # Base case 2:
    # An array of a single element is always palindrome -> return True
```

```
# Base case 3:
# If the first and last elements of arr are not identical,
# the array is not palindrome -> return False

# If we get here it's because the first and last
# elements are identical, therefore:
# Recursive call:
# Call ispalindrome() dropping the first and last
# elements of arr
```

Exercise 3: Files and Data Structures (35 points)

Later in the course, we will use two basic but important data structures: *dictionaries* and *priority queues*. In this exercise, you will be asked to use both to parse and process text.

To test your code, you'll need some text dump. Project Gutenberg (https://www.gutenberg.org/) has thousands of books in various formats, including plain text. For example, here you can find the full text of Alice in Wonderland: https://www.gutenberg.org/files/11/11-0.txt. You don't need to parse the entire book — a chapter or two are sufficient.

1. The first function you need to write takes a filename as input (`fname`) and returns a list containing the words that appear in the text. Only the words, no numbers, no punctuation. The words must be stored lowercase. Use the function `str.lower()` to make a string lowercase (see https://docs.python.org/3.5/library/stdtypes.html#str.lower). Use the function `str.translate()` (see https://docs.python.org/3.5/library/stdtypes.html#str.translate) to remove punctuation. To split a string into words, use `str.split()` (see https://docs.python.org/3.5/library/stdtypes.html#str.split). Treat the apostrophe as punctuation, so "I'm" would become "I m". Also, consider words like "rabbit-hole" in the original text as two separated words: "rabbit" and "hole".

```
def file_to_wordlist(fname):
    wordlist = []
    # your code here
    return wordlist
```

2. The second function you need to write takes the word list generated by the previous function and returns a dictionary (see https://docs.python.org/3.5/tutorial/datastructures.html#dictionaries) in which every word is associated with the number of times it appears in the word list. On StackOverflow you might find solutions that suggest to use `Counter`. Please avoid using that approach, because the point of this exercise is to learn what a dictionary is and does.

```
def wordlist_to_wordfreq(wordlist):
    wordfreq = {}
    # your code here
    return wordfreq
```

3. The third and last function you must write takes the word frequency dictionary, stores it in a priority queue, and returns a sorted word list in increasing order of frequency. Priority queues are often implemented with a data structure called *heap* (see https://docs.python.org/3.5/library/heapq.html), which is what you will use to write this function. Have a look in particular at the `heapsort()` example at https://docs.python.org/3.5/library/heapq.html#basic-examples.

```
import heapq
def wordfreq_to_wordpriority(wordfreq):
    wordpriority = []
    # your code here
    return [heapq.heappop(wordpriority) for i in range(len(wordpriority))]
```

Example

As example, let's take the lyrics of "Bohemian Rhapsody" by Queen. The function `file_to_wordlist('bohemian_rapsody.txt')` returns an array

```
[ 'is', 'this', 'the', 'real', 'life', 'is', 'this', 'just', 'fantasy',
  'caught', 'in', 'a', 'landslide', 'no', 'escape', 'from', 'reality', ... ]
```

The function call `wordlist_to_wordfreq(file_to_wordlist('bohemian_rapsody.txt'))` returns

```
{'all': 4, 'just': 8, 'anyway': 3, 'go': 16, 'scaramouch': 2, 'had': 1,
 'him': 3, 'late': 1, 'to': 9, 'easy': 4, 'has': 2, 'skies': 1,
```

The third function returns:

```
[(1, 'aching'), (1, 'again'), (1, 'against'), (1, 'anyone'), (1, 'as'),
 (1, 'aside'), (1, 'at'), (1, 'away'), (1, 'back'), (1, 'because'),
 (1, 'beelzebub'), (1, 'been'), (1, 'begun'), (1, 'behind'), (1, 'body'),
 ...
 (6, 'will'), (7, 'and'), (7, 'mama'), (8, 'just'), (8, 'the'), (9, 'no'),
 (9, 'to'), (11, 'a'), (11, 'i'), (13, 'let'), (13, 'you'), (16, 'go'), (18, 'me')]
```

Implementation Notes

The books in Project Gutenberg are encoded in UTF8, which is more cumbersome to parse than plain ASCII. Since the point of the exercise is not to make parsing difficult, but to review data structures and some Python syntax, focus your text parsing on ASCII text.

You can either take text that is already in ASCII format, or, under Linux and Mac, you can convert UTF8 to ASCII with this command typed in the terminal (the $ indicates the Bash command prompt, it's not part of the command itself):

```
$ iconv -f UTF8 -t ASCII -c alice_utf8.txt > alice_ascii.txt
```

The command above assumes that the text you want to convert is stored in `alice_utf8.txt`. The option `-c` ignores any character in the UTF8 file that cannot be converted to ASCII. The conversion won't be perfect, but it's not a big deal.